

Bresenham Line Drawing Theory (Easy Step-by-Step)

This note explains the theory behind line drawing in:

- 02_bresenham_line.py

The goal is to make the algorithm easy to understand and easy to trace by hand.

1) What Problem Are We Solving?

We want to draw a straight line between two points on a pixel grid.

A real mathematical line is continuous, but the screen is made of discrete pixels.

At each step, we must choose which pixel is closest to the ideal line.

2) Why Bresenham Instead of $y = mx + c$?

The naive method uses floating-point operations (division and multiplication) for many pixels.

Bresenham is better because:

1. It uses integer arithmetic only.
 2. It is faster and stable.
 3. It is ideal for raster displays and graphics programming.
 4. It avoids repeated floating-point rounding errors.
-

3) Core Idea (Decision Parameter)

For each x-step (or y-step in steep lines), there are two candidate pixels:

1. Move straight in major axis (E or N).
2. Move diagonally (NE, SE, etc.).

① start, end ✓
②

A decision parameter p tells us which candidate is closer to the true line.

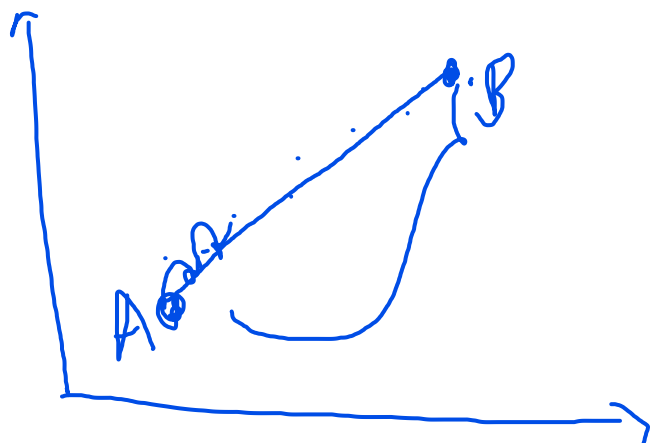
For gentle slope ($\text{abs}(\text{slope}) \leq 1$):

- $dx = \text{abs}(x_2 - x_1)$
- $dy = \text{abs}(y_2 - y_1)$
- $p = 2*dy - dx$

DDA

Update rule each step:

1. If $p < 0$:
 - move x only
 - $p = p + 2*dy$
2. Else:



- move x and y
- $p = p + 2*(dy - dx)$

4) Step-by-Step Algorithm (All Octants)

Real lines can go in any direction. The implementation supports all octants.

Step 1: Inputs

Given start (x_1, y_1) and end (x_2, y_2)

Step 2: Compute deltas and directions

- $dx = \text{abs}(x_2 - x_1)$
- $dy = \text{abs}(y_2 - y_1)$
- $sx = +1$ if $x_2 > x_1$ else -1
- $sy = +1$ if $y_2 > y_1$ else -1

Step 3: Choose major axis

1. If $dx \geq dy$ (gentle line):
 - step mainly in x
2. Else (steep line):
 - step mainly in y

Step 4: Initialize p

1. Gentle case: $p = 2*dy - dx$
2. Steep case: $p = 2*dx - dy$

Step 5: Iterate

At each iteration:

1. Plot current pixel.
2. Use p to choose straight or diagonal move.
3. Update p with integer additions.
4. Continue until destination reached.

5) Worked Example (Gentle Slope)

Draw line from $(2, 3)$ to $(10, 7)$

- $dx = 8$
- $dy = 4$
- $p_0 = 2*4 - 8 = 0$

Step	Pixel (x, y)	p before	Move	p after
------	--------------	----------	------	---------

Step	Pixel (x, y)	p before	Move	p after
0	(2, 3)	0	start	0
1	(3, 4)	0	diagonal	-8
2	(4, 4)	-8	x only	0
3	(5, 5)	0	diagonal	-8
4	(6, 5)	-8	x only	0
5	(7, 6)	0	diagonal	-8
6	(8, 6)	-8	x only	0
7	(9, 7)	0	diagonal	-8
8	(10, 7)	-8	x only/final	0

cas 002



$$P_{k+1} = P_k + 2dy - 2dx$$

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k + 1$$

Pattern here alternates because slope is 0.5.

6) Steep Lines ($\text{abs}(\text{slope}) > 1$)

If $dy > dx$, we step in y as the major axis.

The same idea applies, but x and y roles are swapped in update logic.

This is why the code has two branches:

1. Gentle branch: $dx \geq dy$
2. Steep branch: $dy > dx$

7) Tricky Scenarios and How Bresenham Handles Them

1. Horizontal line ($dy = 0$):
 - p starts negative, only x changes.
2. Vertical line ($dx = 0$):
 - steep branch, only y changes.
3. Negative slope:
 - sy becomes -1, so y decreases correctly.
4. Right-to-left line:
 - sx becomes -1, so x decreases correctly.
5. Single-point line:
 - start equals end, result is one pixel.

8) Time Complexity

- Time: $O(\max(dx, dy))$
- Space: $O(\max(dx, dy))$ if storing all points in a list

9) Mapping Theory to Your Python File

In 02_bresenham_line.py:

1. bresenham_line(...)
 - generates all line pixels.
2. bresenham_line_with_trace(...)
 - gives step-by-step debugging info (p values and decisions).

So the file is not only drawing lines, it is also a teaching tool to observe the decision parameter in action.

10) Quick Pseudocode

```
function bresenham_line(x1, y1, x2, y2):
    dx = abs(x2-x1), dy = abs(y2-y1)
    sx = sign(x2-x1), sy = sign(y2-y1)
    x, y = x1, y1
    plot(x, y)

    if dx >= dy:
        p = 2*dy - dx
        repeat dx times:
            if p < 0:
                x = x + sx
                p = p + 2*dy
            else:
                x = x + sx
                y = y + sy
                p = p + 2*(dy-dx)
            plot(x, y)
    else:
        p = 2*dx - dy
        repeat dy times:
            if p < 0:
                y = y + sy
                p = p + 2*dx
            else:
                y = y + sy
                x = x + sx
                p = p + 2*(dx-dy)
            plot(x, y)
```

11) Practical Use Cases

1. 2D drawing tools and CAD basics
2. Wireframe rendering
3. Grid and map overlays

12) Short Revision Summary

1. Bresenham chooses nearest pixel at each step.
2. Decision parameter p tracks line error.
3. Uses only integer additions/subtractions.
4. Handle all directions with s_x , s_y and gentle/steep branching.
5. Fast, classic, and still important in graphics fundamentals.